C Basics

Mahir Labib Dihan
Lecturer, CSE, BUET

Table of Contents

1. Data Types and Variables
2. Input and Output
3. Operators and Expressions
4. Type Conversion and Casting
5. Practical Examples
6. Problem Solving Methodology

Data Types and Variables

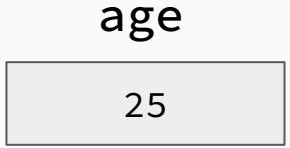
What is a Variable?

Definition

A variable is a **named location in memory** that stores data which can be modified during program execution.

Every variable has:

- A name (identifier)
- A type (what kind of data)
- A size (how much memory)
- A value (the actual data)



Variable storing age value (integer)

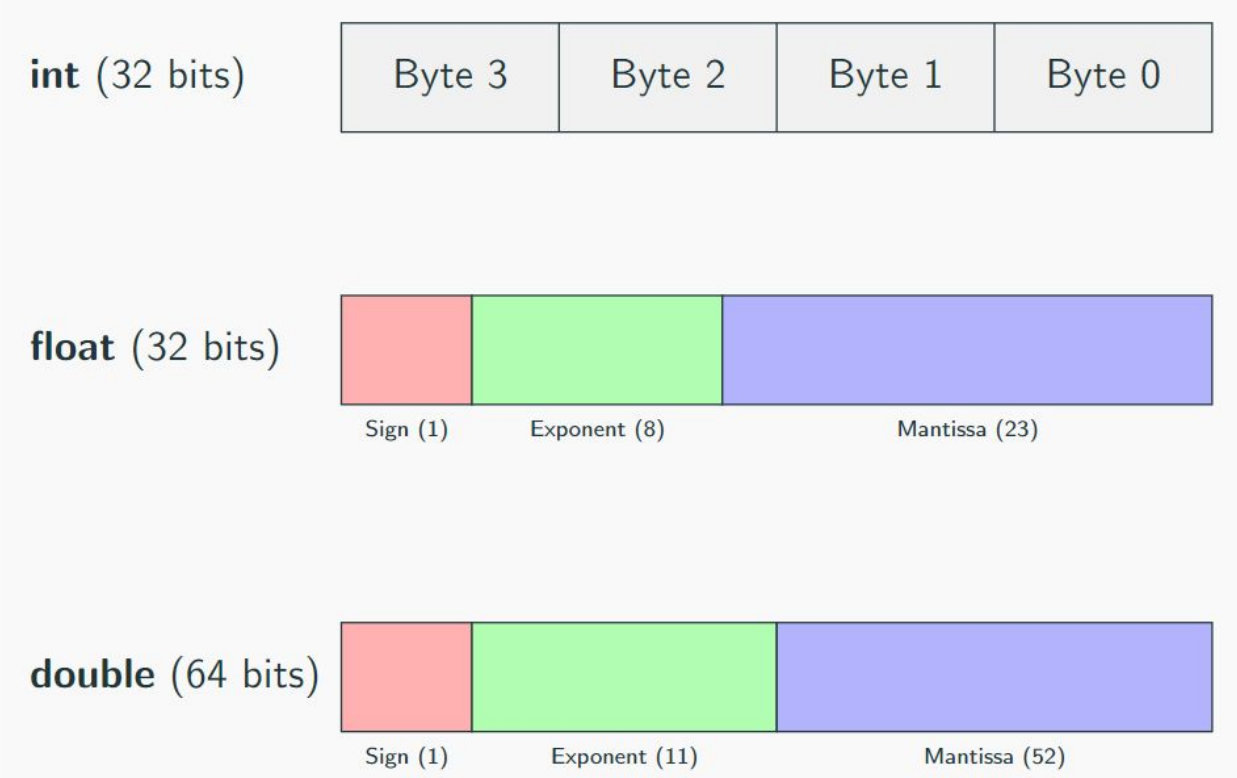
Basic Data Types in C

Type	Example	Size	Range	Format
`char`	`'A', 'z', '$'`	1 byte	-128 to +127	`%c`
`int`	`32, -45, 0`	4 bytes	±2.1 × 109	`%d, %i`
`float`	`3.14, -0.5`	4 bytes	±3.4 × 1038	`%f`
`double`	`3.14159265`	8 bytes	±1.7 × 10308	`%lf`

Definition

- **int:** Age, Count of items, Year
- **float:** Price, Temperature, Height
- **double:** GPS Coordinates, Scientific constants
- **char:** Grade (’A’, ’B’), Gender (’M’, ’F’)

Memory Representation



Extended Data Types

Type	Size	Range	Format Code
unsigned char	8 bits, 1 byte	0 to 255	"%hhu" (Extension is performed when "%u" or "%d" is used)
short int or short	16 bits, 2 bytes	-32,768 to 32,767	"%hd" "%hi"
unsigned short int or unsigned short	16 bits, 2 bytes	0 to 65,535	"%hu"
unsigned int or unsigned	32 bits, 4 bytes	0 to 4,294,967,295	"%u"
long int or long	32 bits, 4 bytes	-2,147,483,648 to +2,147,483,647	"%ld" "%li"
unsigned long int or unsigned long	32 bits, 4 bytes	0 to 4,294,967,295	"%lu"
long long int or long long	64 bits, 8 bytes	-9223372036854775808 to +9223372036854775808	"%lld" "%lli"
unsigned long long int or unsigned long long	64 bits, 8 bytes	0 to 18446744073709551615	"%llu"

Floating Point Precision

Question: What will this code print?

```
float a = 0.1;
if (a == 0.1) {
     printf("Equal");
} else {
     printf("Not Equal");
}
```

Answer: `Not Equal`

Why? 0.1 is a double by default. float a stores an approximation.

- 0.1 (double) ≈ 0.1000000000000000555
- 0.1 (float) ≈ 0.10000001490116119 38

Variable Naming Rules

Valid Characters

Letters `(A-Z, a-z)`, digits `(0-9)`, underscore `(_)`

Rules:

- Must start with letter or _
- Case sensitive
- No spaces allowed
- Cannot use C keywords
- Usually ≤ 31 characters

Examples:

- ✓ total score
- ✓ value1
- ✗ 2nd_value
- ✗ int (keyword)
- ✗ my age

List of all Keywords in C Language

Keywords in C Programming			
auto	break	case	char
const	continue	default	do
double	else	enum	extern
float	for	goto	if
int	long	register	return
short	signed	sizeof	static
struct	switch	typedef	union
unsigned	void	volatile	while

Declaring and Initializing Variables

Declaration: Tells the compiler to reserve memory

```
int count;
float price;
double distance;
char grade;
```

Initialization: Assign initial value

```
int count = 10;
float price = 25.50;
double distance = 12345.6789;
char grade = 'A';
```

Multiple declarations:

```
int x, y, z;
float length = 10.5 , width = 5.2 , area ;
```

Common Mistake: Uninitialized Variables

Mistake

```
int x;           // Uninitialized - contains garbage value
int y = 5;
int sum = x + y; // sum will be garbage + 5 = garbage!
```

Why? Uninitialized variables contain whatever was previously in that memory location.

Good Practice

```
int x = 0;       // Initialize to 0
int y = 5;
int sum = x + y; // sum = 5 (predictable)
```

ASCII Character Codes

Characters stored as numbers (0-127)

- ‘A’ to ‘Z’: 65 to 90
- ‘a’ to ‘z’: 97 to 122
- ‘0’ to ‘9’: 48 to 57
- Space: 32

Example:

```
char ch = 'A';
printf ("%c", ch); // Prints: A
printf ("%d", ch); // Prints: 65
```

ASCII Table (Printable Characters)

Dec	Ch	Dec	Ch	Dec	Ch	Dec	Ch
32	Space	56	8	80	P	104	h
33	!	57	9	81	Q	105	i
34	"	58	:	82	R	106	j
35	#	59	;	83	S	107	k
36	$	60	¡	84	T	108	l
37	%	61	=	85	U	109	m
38	&	62	¿	86	V	110	n
39	'	63	?	87	W	111	o
40	(	64	@	88	X	112	p
41	)	65	A	89	Y	113	q
42	*	66	B	90	Z	114	r
43	+	67	C	91	[	115	s
44	,	68	D	92	\	116	t
45	-	69	E	93	]	117	u
46	.	70	F	94	^	118	v
47	/	71	G	95	_	119	w
48	0	72	H	96	‘	120	x
49	1	73	I	97	a	121	y
50	2	74	J	98	b	122	z
51	3	75	K	99	c	123	{
52	4	76	L	100	d	124	—
53	5	77	M	101	e	125	}
54	6	78	N	102	f	126	~
55	7	79	O	103	g	127	DEL

Input and Output

Output with printf()

Syntax: `printf(format_string, variables);`

```
int count = 5;
float price = 10.50;
char grade = 'A';

printf("Count: %d\n", count);
printf("Price: %f\n", price);
printf("Grade: %c\n", grade);

// Multiple values
printf("Item count: %d, Total Price: %.2f\n", count, price);
```

Output:

```
Count : 5
Price : 10.500000
Grade : A
Item count : 5 , Total Price : 10.50
```

Format Specifiers

Specifier	Type	Example
`%d or %i`	int	`printf("%d", 100);`
`%f`	float	`printf("%f", 3.14);`
`%lf`	double	`printf("%lf", 3.14159);`
`%c`	char	`printf("%c", 'A');`
`%.2f`	float (2 decimals)	`printf("%.2f", 3.14159);`

```
float pi = 3.14159265;
printf("%.2f\n", pi); // Output: 3.14
printf("%.4f\n", pi); // Output: 3.1416
printf("%10.2f\n", pi); // Output: "3.14" (width=10)
```

Common Mistake: Wrong Format Specifier

Mistakes

```
double pi = 3.14159;
scanf("%f", &pi);     // WRONG! Use %lf for double

float value = 2.5;
printf("%d", value); // WRONG! Prints garbage

int count = 10;
printf("%f", count); // WRONG! Prints garbage
```

Correct

```
double pi;
scanf("%lf", &pi);        // CORRECT for double
float value = 2.5;
printf("%f", value);      // CORRECT
int count = 10;
printf("%d", count);      // CORRECT
```

Input with scanf()

Syntax: `scanf(format string, &variables);`

Note the & (address-of operator)!

```c
int quantity;
float weight;
char category;

printf("Enter quantity: ");
scanf("%d", &quantity);

printf("Enter weight (kg): ");
scanf("%f", &weight);

printf("Enter category (A/B/C): ");
scanf(" %c", &category);  // Space before %c important!

printf("Qty: %d, Wt: %.1f, Cat: %c\n",
       quantity, weight, category);
```

Common Mistake: Forgetting & in scanf

Mistake

```
int age ;
scanf("%d", age); // WRONG! Missing & operator
```

Why? scanf needs the address of the variable to store the value.

Correct

```
int age ;
scanf("%d", &age); // CORRECT!
```

Complete Input/Output Example: The Problem

Task: Calculate the area of a rectangle.

Requirements:

- Ask user for Length (in meters)
- Ask user for Width (in meters)
- Calculate Area = Length × Width
- Display result in square meters

Complete Input/Output Example: The Code

Area Calculator

```
#include <stdio.h>

int main() {
      float length, width, area ;

      printf("Enter length (m): ");
      scanf("%f", &length);

      printf("Enter width (m): ");
      scanf("%f", &width);

      area = length * width;

      printf("Area = %.2f sq.meters\n", area);
      return 0;
}
```

Operators and Expressions

Arithmetic Operators

Operator	Operation	Algebraic	C Expression
`+`	Addition	`f + 7`	`f + 7`
`-`	Subtraction	`p - c`	`p - c`
`*`	Multiplication	`b × m`	`b * m`
`/`	Division	`x / y`	`x / y`
`%`	Modulus (remainder)	`r mod s`	`r % s`

Real Life Example

Average Speed: $Speed = \frac{\text{Distance}}{\text{Time}}$

```
speed = distance / time;
```

Integer vs Floating-Point Division

Important Distinction!

The result type depends on operand types

Integer Division:

```
int a = 7;
int b = 3;
int c = a / b;
// c = 2 (truncated!)

int d = 5 / 2;
// d = 2
```

Floating-Point Division:

```
float a = 7.0;
float b = 3.0;
float c = a / b;
// c = 2.333333

float d = 5.0 / 2.0;
// d = 2.5
```

Mixed operations:

```
float result = 5.0 / 2;     // 2.5 (one operand is float)
float result2 = 5 / 2.0;    // 2.5 (one operand is float)
float result3 = 5 / 2;      // 2.0 (both operands are int!)
```

Division by Zero

Question: What happens if you divide by zero in C?

- `int x = 5 / 0;`
- `float y = 5.0 / 0.0;`

Answer:

- **Integer Division by Zero:** Usually causes **a runtime error** (program crashes).
- **Floating-Point Division by Zero:** Results in **Infinity** (`inf`) or **NaN** (Not a Number).

Common Mistake: Integer Division

Mistake

```
int sum = 47;
int count = 5;
float average = sum / count ; // average = 9.0 (WRONG!)
```

Why? Integer division happens first (47/5 = 9), then result is converted to float.

Correct Solutions

```
// Solution 1: Cast one operand
float average = (float) sum / count;  // 9.4 (CORRECT)

// Solution 2: Make dividend float
float average = sum / (float) count;  // 9.4 (CORRECT)

// Solution 3: Use float variables
float sum_f = 47.0;
float average = sum_f / count;        // 9.4 (CORRECT)
```

Modulus Operator

Modulus (%): Returns the remainder after division

Only works with integers!

```
int a = 17 % 5; // a = 2 (17 = 3*5 + 2)
int b = 20 % 4; // b = 0 (20 = 5*4 + 0)
int c = 7 % 10; // c = 7 (7 = 0*10 + 7)
```

Application: Extract Digits

```
int number = 4567;
int last_digit = number % 10;          // 7
int remaining = number / 10;           // 456
int second_last = remaining % 10;      // 6
```

Operator Precedence

Precedence	Operators	Associativity
Highest	`( )`	Left to right
↓	`*, /, %`	Left to right
Lowest	`+ , −`	Left to right

Examples:

$5 + 6 \times 7 - 8/2 = 5 + 42 - 4 = 43$

$5 \times 4/3 = 20/3 = 6$ (integer division)

$(5 + 6) \times 7 = 11 \times 7 = 77$

Best Practice

Use parentheses to make your intentions clear!

Figuring Out Precedence

What is the value of x?

```
int x = 2 * 3 / 4 + 4 / 4 + 8 - 2 + 5 / 8;
```

Step-by-Step Evaluation:

1. $2 * 3 \rightarrow 6$
2. $6 / 4 \rightarrow 1$ (integer division)
3. $4 / 4 \rightarrow 1$
4. $5 / 8 \rightarrow 0$ (integer division)
5. $1 + 1 + 8 - 2 + 0 \rightarrow 8$

Result: $x = 8$

Increment and Decrement Operators

Post-increment: `var++`

```
int x = 5;
int y = x++;
// y = 5, x = 6
// (use then increment)
```

Pre-increment: `++var`

```
int x = 5;
int y = x++;
// y = 5, x = 6
// (use then increment)
```

Post-decrement: `var--`

```
int x = 5;
int y = x--;
// y = 5, x = 4
// (use then decrement)
```

Pre-decrement: `--var`

```
int x = 5;
int y = --x;
// y = 4, x = 4
// (decrement then use)
```

```
int count = 10;
count++;         // count = 11 (standalone use: same result)
++count ;        // count = 12 (standalone use: same result)
```

Some Increment/Decrement

What is the output of this code?

```
int b = 10;
int c = 5;
int a = b+++--c;
printf("a = %d, b = %d, c = %d\n", a, b, c);
```

Analysis:

- The compiler parses `b+++--c` as `(b++)+(--c)`.
- `b++` (Post-increment): Uses original value **10**, then increments b to **11**.
- `--c` (Pre-decrement): Decrements **c** to **4**, then uses value **4**.
- `a = 10 + 4 = 14`.

Output: `a = 14, b = 11, c = 4`

Warning

Avoid writing such complex expressions! They are hard to read and can lead to undefined behavior if you modify the same variable multiple times (e.g., `i = i++ + 1;`).

Assignment Operators (Shorthand)

Long Form	Shorthand	Meaning
`x = x + 5`	`x += 5`	Add 5 to x
`x = x - 3`	`x -= 3`	Subtract 3 from x
`x = x * 2`	`x *= 2`	Multiply x by 2
`x = x / 2`	`x /= 2`	Divide x by 4
`x = x % 3`	`x %= 3`	x modulo 3

```
float price = 100.0
price += 50.0   // price = 150.0
price *= 2          // price = 300.0
price /= 3          // price = 100.0

int count = 10;
count -= 3          // count = 7
count %= 4          // count = 3
```

Type Conversion and Casting

Automatic Type Conversion

Promotion: Lower type → Higher type (automatic)

char → int → float → double

```
int x = 9;
float y = x; // OK, no warning (9.0)

char ch = 'A';
int code = ch; // OK, code = 65

float f = 5.7;
int i = f; // WARNING! (truncates to 5)
```

Mixed Operations

In an expression with mixed types, lower types are automatically promoted to match the highest type present.

Explicit Type Casting

Syntax: (type) expression

```
float price = 250.8;
int rounded = (int) price;       // rounded = 250 (no warning)

int a = 5, b = 2;
float result = (float) a / b;    // 2.5 (correct!)
float wrong = a / b;             // 2.0 (integer division first!)

double pi = 3.14159265;
float pi_float = (float) pi;     // Explicit conversion
```

Average Calculation:

```
int sum = 47;
int count = 5;
float average = (float) sum / count; // 9.4 (correct!)
```

Type Conversion Examples

```
// Example 1
float x = 5 / 3;           // x = 1.0 (int division, then convert)
float y = 5.0 / 3;         // y = 1.666667 (float division)

// Example 2
int a = 5, b = 2;
float c = a / b; // c = 2.0 (wrong!)
float d = (float) a / b; // d = 2.5 (correct!)

// Example 3: Character arithmetic
char ch = 'A';
int next = ch + 1; // next = 66
char next_ch = ch + 1; // next_ch = 'B'

// Example 4: Mixed expression
int i = 10;
float f = 5.5;
double d = i + f; // d = 15.5 (i promoted to float)
```

Practical Examples

Example 1: Temperature Conversion

$$F = 9/5\ C + 32, K = C + 273.15$$

```
float celsius, fahrenheit, kelvin;

printf("Enter temperature in Celsius: ");
scanf("%f", &celsius);

// Note: 9.0/5.0 to ensure float division
fahrenheit = (9.0 / 5.0) * celsius + 32.0;
kelvin = celsius + 273.15;

printf("%.2f C = %.2f F\n", celsius, fahrenheit);
printf("%.2f C = %.2f K\n", celsius, kelvin);
```

Example 2: Character Operations

```c
char ch;

printf("Enter a capital letter: ");
scanf("%c", &ch);

printf("Character: %c\n", ch);
printf("ASCII code: %d\n", ch);
printf("Lowercase: %c\n", ch + 32);
printf("Next letter: %c\n", ch + 1);

// Convert uppercase to lowercase
char lowercase = ch + ('a' - 'A'); // or ch + 32
printf("Converted to lowercase: %c\n", lowercase);
```

Hint: ‘A’ to ‘Z’ differ from ‘a’ to ‘z’ by exactly 32.

Example 3: Reverse a 4-digit Number

```
int number, digit1, digit2, digit3, digit4, reversed;

printf("Enter a 4-digit number: ");
scanf("%d", &number);

// Extract digits from right to left
digit4 = number % 10; // Last digit
digit3 = (number / 10) % 10;
digit2 = (number / 100) % 10;
digit1 = (number / 1000) % 10; // First digit

// Reconstruct in reverse
reversed = digit4*1000 + digit3*100 + digit2*10 + digit1 ;

printf("Original: %d, Reversed: %d\n", number, reversed);
```

Example 4: Swap Two Variables

Method 1: Using a temporary variable

```
int a = 10, b = 20, temp;
temp = a;
a = b;
b = temp;
// Now : a = 20 , b = 10
```

Method 2: Without temporary variable (arithmetic)

```
int a = 10, b = 20;
a = a + b; // a = 30
b = a - b; // b = 10
a = a - b; // a = 20
// Now: a = 20, b = 10
```

Note

Method 2 can cause overflow with large numbers. Method 1 is safer.

Problem Solving Methodology

Problem Solving Process

1. **State the problem clearly**
   - What is the objective?
   - What are the constraints?

2. **Describe input/output**
   - What data is needed?
   - What results should be produced?

3. **Work the problem by hand**
   - Use concrete example values
   - Verify your understanding

4. **State the problem clearly**
   - What is the objective?
   - What are the constraints?

5. **Describe input/output**
   - What data is needed?
   - What results should be produced?

Acknowledgement

- Mohammad Sadat Hossain, Lecturer, CSE, BUET